Project Blueprint: TaskCraft (Task Management System)

Target Stack: React, Tailwind CSS, Component Libraries (MUI or Shadcn UI), Document-Based NoSQL (MongoDB/Firestore), and Express/Node.js REST API with JWT.

1. Core Feature Specifications

To build a high-quality product, we focus on a clean, centralized list interface backed by robust validation, declarative route guarding, and smooth URL-driven modal/drawer interactions.

A. List View Layout (Main Dashboard)

Instead of disorganized cards, TaskCraft features a central **List View Table** designed using a component library (such as **Material UI (MUI)** or **Shadcn UI**).

- **The Task Grid Table:** A responsive table containing:
  - **Quick Status Checkbox:** An interactive checkbox or custom switch right on the row that allows users to instantly toggle the task status between "Done" and "To Do" with a single click, instantly updating the database.
  - **Status Badge:** e.g., "To Do" (slate grey), "In Progress" (sky blue), "Done" (emerald green).
  - **Task Title & Description:** Highlighted, easily readable text.
  - **Priority Indicator:** e.g., "High" (rose red badge), "Medium" (amber yellow), "Low" (emerald).
  - **Due Date / Overdue Flag:** Formatted due date. If the current date exceeds the due date and the task is incomplete, a crimson **"Overdue"** badge glows next to the date.
  - **Assignee:** Displays user name/avatar.
  - **Tags:** Dynamic inline badges (e.g., "Frontend", "Backend", "Bug").
- **URL-Driven Drawer / Dialog Interaction (Deep Linking):**
  - When a user clicks on a task row (or an "Edit" button inside the row), the application changes the browser URL path (e.g., navigating to /dashboard/task_10029 or appending a query parameter like /dashboard?taskId=task_10029).
  - The application detects this URL parameter (using React Router's useParams or useSearchParams) and automatically slides open a responsive **Detail Drawer** (from the right side of the screen) or pops up an accessible **Dialog Modal**.
  - **Inside the Drawer/Dialog:** Users can view detailed history, update any fields (title, description, tags, assignee), check off custom subtasks, or permanently **Delete** the task.
  - **Closing the Drawer/Dialog:** Clicking outside, pressing Escape, or clicking "Cancel" updates the URL back to the clean dashboard path (e.g., /dashboard), automatically closing the view. This makes your application state highly shareable and robust!

B. User Validation Requirements (Form Fields)

To guarantee data integrity and protect system resources, enforce strict validation constraints on the client and server.

Sign-Up Validation

- **Full Name:** Required, minimum 2 characters, alphabetic characters only.
- **Email Address:** Required, must conform to standard email regex syntax (^[^\s@]+@[^\s@]+\.[^\s@]+$).
- **Password:** Required, minimum 8 characters, must contain at least one uppercase letter, one lowercase letter, one numeric digit, and one special character (e.g., @, $, !, %, *, ?, &).
- **Confirm Password:** Required, must strictly match the password field.

Sign-In Validation

- **Email Address:** Required, standard email syntax validation.
- **Password:** Required, non-empty. Display a generalized error message upon validation failure (e.g., *"Invalid email or password credentials"*) to protect user accounts from brute-force attempts.

2. JWT Authentication & Route Guarding

We utilize a clean, single-token JSON Web Token (JWT) workflow to authorize user sessions and protect application routes.

2.1 Single-Token Authentication Lifecycle

1. **User Sign-In / Register:** The user submits validated credentials.
2. **Token Generation:** The Node.js server verifies the credentials, signs a JWT access token containing secure claims (User ID, Email), and sends it back to the client as a JSON payload.
3. **Session Memory Storage:** The React client receives the token and stores it securely in memory (such as within your React/Redux global context state).
4. **Authorized Requests:** For every subsequent task request, the client attaches this token to the HTTP header:
   Authorization: Bearer <Your_JWT_Token>

2.2 Protecting Routes Using React Router <Outlet />

To block unauthenticated users from entering protected sections of the application, we use declarative route guarding with React Router's <Outlet />.

The Protected Route Pattern

An authentication guard component acts as a gatekeeper. If a valid token is present in the

application state, it renders an <Outlet /> which acts as a placeholder for child dashboard routes. If the user is unauthenticated, they are redirected to /login using a <Navigate /> element.

Client Routing Configuration

Your router should wrap authenticated pages in a parent route managed by this guard component, nesting the task detail path to trigger the URL-based Drawer:

```javascript
// Example Router setup for developer reference:
<Routes>
  {/* Public Routes */}
  <Route path="/login" element={<LoginPage />} />
  <Route path="/register" element={<RegisterPage />} />

  {/* Protected Application Routes */}
  <Route element={<ProtectedRoute />}>
    <Route path="/dashboard" element={<DashboardLayout />}>
      {/* List view is shown by default */}
      <Route index element={<TaskListView />} />
      {/* Nested path matching the URL task ID to open the drawer
*/}
      <Route path=":taskId" element={<TaskListView />} />
      <Route path="profile" element={<UserProfileView />} />
    </Route>
  </Route>
</Routes>
```

3. NoSQL Database Schema Design

This document-oriented schema provides high-performance data reading, allowing tags and task creators to be stored without requiring multi-table SQL joins.

3.1 Users Collection (/users)

```json
{
  "_id": "usr_alpha_1",
  "name": "Jane Doe",
  "email": "jane@taskcraft.com",
  "passwordHash": "$2b$12$K392jsdh...892hjdS7yS",
  "avatarUrl": "[https://api.dicebear.com/avatar/jane.svg]",
  "createdAt": "2026-07-15T09:00:00Z"
```

```
}
```

3.2 Tasks Collection (/tasks)

```JSON
{
  "_id": "task_10029",
  "title": "Configure Application Routing",
  "description": "Set up protected route guards and deep-linked
drawer interfaces.",
  "status": "In Progress",
  "priority": "High",
  "dueDate": "2026-07-20T18:30:00Z",
  "tags": ["Frontend", "Routing"],
  "assignedTo": "usr_alpha_1",
  "createdBy": "usr_beta_2",
  "subtasks": [
    { "id": "sub_1", "text": "Create ProtectedRoute component",
"isCompleted": true },
    { "id": "sub_2", "text": "Set up URL-driven Drawer with
params", "isCompleted": false },
    { "id": "sub_3", "text": "Test redirect loop logic",
"isCompleted": false }
  ],
  "createdAt": "2026-07-15T09:00:00Z"
}
```

4. API Endpoint Layout

All backend business processes are exposed via REST endpoints. Authenticated endpoints require a verified bearer token in the Authorization header.

4.1 Public Authentication Endpoints

- POST /auth/register (Registers user, hashes password, returns access token)
- POST /auth/login (Validates user, signs and returns access token)

4.2 Protected Task Endpoints (requireAuth Middleware)

- GET /tasks (Retrieves tasks. Filters by status, tags, and search keywords using query strings)
- POST /tasks (Creates a task document)
- PUT /tasks/:id (Updates task fields, completion states, assignees, and subtask arrays)
- DELETE /tasks/:id (Deletes a task record)

5. Deployment & Production Specifications

To transition TaskCraft successfully from local environments to production servers, the application architecture must fulfill specific infrastructure behaviors:

A. Environment-Aware Configuration

- The frontend client must dynamically resolve its base API request path by reading environment-specific build configurations (e.g., using `import.meta.env` or process variable injection). It should transition from `localhost` targets in development to public secure domain targets in production without code edits.
    - **Development:** `VITE_API_URL=http://localhost:5000`
    - **Production:** `VITE_API_URL=https://taskcraft-backend.onrender.com`

B. Cross-Origin Security Policies (CORS)

The Express server must act as a gateway that enforces CORS rules. In production, it must dynamically authorize request headers and cookies coming *only* from the specified secure production URL where the frontend is hosted, preventing unauthorized external websites from communicating with the server.

C. Express Backend Deployment on Render

- **Host Type:** Render Web Service (Node.js environment).
- **Build Command:** `npm install`
- **Start Command:** `node server.js` (or paths to your production startup scripts).
- **Required Production Environment Variables:**
    - `PORT`: Automatically set by Render.
    - `NODE_ENV`: Set to `production`.
    - `DATABASE_URI`: Connection string to the hosted MongoDB Atlas or secure NoSQL cloud database.
    - `JWT_SECRET`: A long, randomly generated secure key for signing tokens.

D. React Frontend Deployment on Vercel

- **Host Type:** Vercel Static Web App.
- **Build Command:** `npm run build` (compiled output folder `dist` or `build`).
- **Output Directory:** `dist`

E. NoSQL Network Security Whitelisting

- The NoSQL database must support dynamic serverless and host cloud IP pools. The DB firewall must be configured to permit secure queries originating from the backend host cluster (e.g., adding `0.0.0.0/0` on MongoDB Atlas) while relying on cryptographically secure connection strings to prevent database breaches.

6. Judging Criteria & Quality Standards

Projects will be evaluated based on execution quality across the following six core dimensions. Developers should prioritize clean logic, proper framework patterns, and consistent user patterns.

A. Design & UI Polish

- **Component Library Standards:** Seamless integration of UI components (e.g., MUI or Shadcn UI). Elements like select lists, priority badges, and input text fields must adopt a uniform aesthetic.
- **Row-Level Interactive Feedback:** The inline status-toggle mechanism (checkbox/switch) must respond instantly on the screen when clicked. It should display responsive visual transitions while sending database updates in the background.
- **Detail Interface Presentation:** The right-sliding Drawer or overlay Dialog must display a clear, logically categorized hierarchy. Forms, update controls, checklist subtasks, and delete buttons must remain easy to read.

B. Responsive Layout Integrity

- **Viewport Adaptability:** The main task grid must scale down nicely for tablet and phone environments. Wide datasets should gracefully wrap, hide low-priority columns, or utilize scrolling containers to avoid spilling out of the screen.
- **Fluid Modal Conversions:** On mobile screen widths, Drawer and Dialog popups must adjust automatically to consume full-screen dimensions, creating a cohesive, touch-friendly mobile interface.
- **Visual Safety Limits:** The application must prevent unexpected text clipping, overlapping input elements, or layout collapses on any screen resolution.

C. Technology Stack & Integration Practices

- **Robust Single-Token Handshake:** Seamless execution of authentication steps. The React interface must properly attach the signed JWT Bearer token to all outgoing requests, while the server must cleanly parse credentials on arrival.
- **Cohesive NoSQL Schema Modeling:** Smart mapping of data collections. Embed list tags and task checklist arrays cleanly inside single records without relying on relational joins.
- **Secure Credential Lifecycle:** Safe password handling protocols. No plain text passwords can touch the database. All credential entry registers must utilize industry-standard server-side hashing libraries (e.g., bcrypt).

D. Directory & Project Structure

- **Separation of Concerns:** Clear physical decoupling of Frontend (client) and Backend (server) code architectures.
- **Structured Client Organization:** Clear organization of React layers (grouping reusable components, page view models, routers, and global helper functions logically in distinct folders).
- **Decoupled Backend Architecture:** Clean MVC setup on the server. Models, control engines, routing endpoints, and check processes must be split into modular, easily discoverable script pathways.

E. Route Architecture & Navigation Security

- **Unyielding <Outlet /> Route Guards:** Safe navigational routing. Unauthenticated users must be fully blocked from entering the workspace paths, with the app redirecting them gracefully back to the credential interfaces.
- **Deep Linked Parameter Parsing:** Robust URL-driven state parsing. Putting a task identifier in the browser path (e.g., /dashboard/task_10029) must read using routing parameters to immediately open the matching detailed workspace view. De-selecting or closing must gracefully restore a clean base path.
- **Clean, Pure Endpoints:** Complete omission of /api prefixes in server routing layouts. Endpoints must read semantic, clean routing paths directly (e.g., /tasks and /auth/register).

F. Middleware Controls & Server Validation

- **Gatekeeping Token Parser Middleware:** Clean configuration of a modular requireAuth step. It must automatically catch unauthorized requests, reject expired payloads, and attach valid decoded user claims to incoming request objects.
- **Robust Input Schema Tests:** Bulletproof validation controls on the server ensuring email formatting conforms to pattern matches, passwords fulfill complexity rules, and confirmation structures align perfectly.
- **Secure Error Boundaries:** Server-side errors must be caught and gracefully formatted into useful client-side messages (such as *"Passwords do not match"* or *"Authentication failed"*), preventing system crashes from being sent directly to the user.

✨ 6. Stretch Goal: Optional Kanban Upgrade

Once you master the standard table layout, consider scaling your TaskCraft system to the next level:

- **The Kanban Experiment:** Create a dynamic board containing three functional status columns: *To Do*, *In Progress*, and *Done*.
- **Interactivity:** Build simple "move" buttons next to each task card, or try using easy drag-and-drop libraries (like @hello-pangea/dnd or dnd-kit) to let users reorganize tasks across columns.
- **Why Build It?** This optional feature teaches you how to map status fields dynamically to column states and design high-fidelity workspace layouts.